AEEDevelopment

<Arithmetic Expression Evaluator >
Software Development Plan
Version <1.0>

Revision History

Date	Version	Description	Author
<2/22/2026>	<1.0>	<SDP including relevant artifcats>	<William Webb>

Table of Contents

1. Introduction 4

1.1 Purpose 4

1.2 Scope 4

1.3 Definitions, Acronyms, and Abbreviations 4

1.4 References 4

1.5 Overview 5

2. Project Overview 5

2.1 Project Purpose, Scope, and Objectives 5

2.2 Assumptions and Constraints 5

2.3 Project Deliverables 5

2.4 Evolution of the Software Development Plan 5

3. Project Organization 5

3.1 Organizational Structure 5

3.3 Roles and Responsibilities 5

4. Management Process 6

4.2 Project Plan 6

4.3 Project Monitoring and Control 6

4.5 Quality Control 7

4.7 Risk Management 7

4.8 Configuration Management 7

5. Annexes 7

Software Development Plan

1. Introduction

1.1 Purpose

The purpose of the *Software Development Plan* is to gather all the information necessary to control the project. It describes the approach to the development of the software and is the top-level plan generated and used by managers to direct the development effort.

The following people use the *Software Development Plan*:

- The **project manager** uses it to plan the project schedule and resource needs, and to track progress against the schedule.
- **Project team members** use it to understand what they need to do, when they need to do it, and what other activities they are dependent upon.

1.2 Scope

This *Software Development Plan* describes the plan to be used by the Arithemtic Expression Evaluator project, including deployment of the product. The details of the individual iterations will be described in the Iteration Plans.
The plans outlined in this document are based upon the product requirements as defined in the *Vision Document*.

Project Scope:

- Development of a Qt C++ desktop application for parsing and evaluating arithmetic expression
- Implementation of core parsing algorithms (tokenization, parsing, evaluation)
- Develop a GUI with advanced visualization features
- Testing and documentation

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
AST	Abstract syntax tree – tree representation of expression structure
PEMDAS	Operator precedence rules
Token	Smallest, meaningful unit in expression string into tokens
Tokenizer/Lexer	Component that converts expression string into tokens
Parser	Component that builds syntax tree from tokens
Evaluator	Component that computes result from syntax tree
Qt	Cross-platform C++ GUI framework
CMake	Cross-platform build system generator
GUI	Graphical user interface
API	Application Programming Interface

QA	Quality Assurance
DevOps	Development Operations
UI/UX	User Interface / User Experience
SDP	Software Development Plan

1.4 References

The following documents are referenced in this Software Development Plan:

1. Project Description Document

EECS 348: Term Project in C++

2. Vision Document

AEEDevelopment Vision Statement
Version 1.0

3. Programming Guidelines

AEEDevelopment Programming standards
Version 1.0

Following Documents are under development

4. Requirements Document

5. Design Document

6. Test Plan

7. Iteration Plan

1.5 Overview

This *Software Development Plan* contains the following information:

Project Overview — provides a description of the project's purpose, scope, and objectives. It also defines the deliverables that the project is expected to deliver along

Project Organization — describes the organizational structure of the project team, including roles, responsibilities, and team member assignments

Management Process — Explains the project schedule, iteration objectives, release strategy, monitoring and control mechanisms, quality control procedures, risk management approach, and configuration management practices.

Annexes — References applicable process guidelines (ex. Programming guidelines and other documentation)

2. Project Overview

2.1 Project Purpose, Scope, and Objectives

Purpose**:** *To develop a C++ program that is fully capable of parsing and evaluating arithmetic expressions which contain the operators -,+,/,*,** and % along with numeric constraints. This program should also be able to handle parentheses so there is a set order of grouping and priority within the program. The program should be able to handle an arithmetic expression with all the requirements listed above as well as be able to work with a larger compiler.*

Deliverables: *The deliverables will include a Vison statement document, a project plan, a requirements document, a programming guidelines document, test cases based on the requirements and design documents.*

2.2 Assumptions and Constraints

Assumptions:

- Team members understand all math operators and PEMDAS order
- Team members have access to the necessary technology to complete/work on the project
- Team members can code in C++ or will learn before writing code
- All of the deadlines given by the professor will remain as scheduled

Constraints:

- Due dates for project iterations
- UPEDU formatting for project plan and certain document formats for requirements and designs
- C++ program language for the code
- Required deliverables for the project like project plan, requirements and design documents, test plan, program, and users manual

2.3 Project Deliverables

Vision Document	Project vision and functionality overview	Week 3
Project Glossary	Definitions of Standards and conventions	Week 3
Programming Guidelines	Coding standards and conventions	Week 4
Software Development Plan	This documentation	Week 4 | 2/22
Software Requirements Specification	Detailed functional and non-functional requirements	Week 7 | 3/15
Iteration Plan 1	Plan for first development iteration	Week 7
Software Architecture Document	System architecture and class design	Week 10 | 4/5
Test Plan	Detailed test cases and expected results	Week 10

Iteration Plan 2	Plan for second development iteration	Week 11
Core Engine Implementation	Tokenizer, Parser, Evaluator classes	Week 12
GUI Implementation	Qt-based user interface	Week 13
Test Cases/results	Detailed test cases and expected results	Week 13
User Manual	End-User Documentation	Wekk 13
Final Application	Compiled application for all platforms	Week 14 | 5/7

Deliverables for each project phase are identified in the Development Case. Deliverables are delivered towards the end of the iteration, as specified in section *4.2.4 of Project Schedule*.

2.4 Evolution of the Software Development Plan

The *Software Development Plan* will be revised prior to the start of each Iteration phase.

Unscheduled revisions may look like:

- Significant changes in scope
- Technical constraints
    - Major schedule adjustments

Revision Process will typically look like the following:

- Change is proposed by any team member
- Project Manager reviews the proposed change
- Team discusses and approves/rejects the change
    - Updated document is committed to version control

3. Project Organization

3.1 Organizational Structure

Roles and Responsibilities

Person	Unified Process for EDUcation Role	Secondary Role	Email
Ritwik Goyal	Product Manager/Team Lead	Software Architect	ritwik.goyal@ku.edu
Jacob Spry	Software Developer/Tester	Software Architect	jacobspry@ku.edu
Andrew Wendling	Software Developers/Testers	Software Architect	awendling01@ku.edu
Ayesha Akbar	Quality Assurance (QA)	UI/UX Designer	ayesha.akbar@ku.edu

Mahanthi Sai	DevOps Engineer	UI/UX Designer	mahathisai.palukuri@ku.edu
William Webb	Software Architect	Product Manager	Wwe3b@ku.edu

Role	Responsibility
Product Manager	Project coordination, progress tracking and reporting, stakeholder communication, final decision making authority
Software Architect	
Software Developer	Implements classes, data structures, algorithm research and selection
Quality Assurance (QA)	Test case design, defect documentation, test execution and reporting
DevOps Engineer	Build system configuration (CMake), CI/CD pipeline setup
UI/UX Designer	GUI development: Mainwindow, tree visualization, step-by-step display, token stream view, history panel
Documentation	Code documentation using Doxygen, Everyone communicating research through teams or discord.

Anyone on the project can perform Any Role activities.

4. Management Process

4.1 Project Plan

4.1.1 Iteration Objectives

The project will be developed using an iterative approach with the following planned iterations, see the *Iteration Plan Document* for more details:

Iteration 0: Project Inception

Iteration 1: Requirements and Design

Iteration 2: Core Engine Implementation

Iteration 3: GUI Development

Iteration 4: Advanced Features and Polish

Iteration 5: Testing and Documentation

4.1.1.1 Releases

This Project will have the following release milestones

Alpha Release (End of Iteration 2):

- Core engine functionality without GUI
- Features
    - Command-line interface for testing
    - Basic expression evaluation
    - Core test cases passing
    -

Beta Release (End of Iteration 3):

- Integrated application with basic GUI
- Features
    - Functional GUI
    - Parse Tree Visualization
    - Basic expression evaluation
    - Most test cases passing

Final Release (End of Iteration 5):

- Tested application and submission
- Features:
- All requirements met
- All test cases passing
    - Complete documentation
    - Builds for Windows, macOS, Linux
    - Professionally packaged

4.1.2 Project Schedule

Phase	Completion Date	Milestones
Inception	Week 4	SDP, Vision, Guidelines Complete
Requirements	Week 7	Requirements Document Complete
Design	Week 10	Design Document Complete
Implementation	Week 12	Core & GUI complete
Testing	Week 13	All Tests passing
Documentation	Week 14	User Manual Complete
Delivery	Week 14	Final Submission

4.2 Project Monitoring and Control

This project will be monitored and controlled through the following mechanisms:

Progress Tracking:

- Weekly team meetings with status updates
- Github project board for task tracking
- Iteration burndown charts

- Milestone completion tracking
- Code commit frequency monitoring

Control Mechanisms:
- Code review requirements before merge
- Iteration evaluation
- Schedule variance tracking
- Scope change control process

Metrics to Track:
- Lines of code completed
- Number of tests written and passing
- Number of open vs closed issues
- Build success rate

4.2.1 Quality Control

Defects will be recorded and tracked as Change Requests, and defect metrics will be gathered (see Reporting and Measurement below).

All deliverables are required to go through the appropriate review process, as described in the Development Case. The review is required to ensure that each deliverable is of acceptable quality, using guidelines and checklists.

Any defects found during review which are not corrected prior to releasing for integration must be captured as Change Requests so that they are not forgotten.

Quality Assurance Process

1. Code quality:

    - All code follows programming guidelines

- Mandatory ode review for pull requests

2. Testing requirements

    - GUI functional testing
    - All 15 valid expressions mus pass
    - All 10 invalid expressions must be caught
    - Test coverage target: 90% minimum

3. Review process

    Code Reviews:

    - At least one team member review all code
    - Reviewers must approve before merge to main

    Document Reviews:

- All documents reviewed by at least two members
- Grammar, formatting, and technical accuracy verified.

Design Review:

- Architecture revew before implementation
- Interface design review
- Algorithm review

4. Defect Tracking
   - Defects logged as GitHub issues
   - Status tracked (in progress, resolved)
   - Metrics collected and reported

4.2.2 Risk Management

Risks will be identified in Inception Phase using the steps identified in the RUP for Small Projects activity "Identify and Assess Risks". Project risk is evaluated at least once per iteration and documented in this table.

Refer to the Risk List Document (Risk-list-v1.0.doc) for detailed information.

4.2.3 Configuration Management

Appropriate tools will be selected which provide a database of Change Requests and a controlled versioned repository of project artifacts.

All source code, test scripts, and data files are included in baselines. Documentation related to the source code is also included in the baseline, such as design documentation. All customer deliverable artifacts are included in the final baseline of the iteration, including executables.

The Change Requests are reviewed and approved by one member of the project, the Change Control Manager role.

Refer to the Configuration Management Plan (EEE-FFF-X.Y.doc) for detailed information.

5. Annexes

The project will follow the UPEDU process.

The following documents provide additional detail and guidance for this project:

1. Programming Guidelines.
2. Risk List
3. Confiugration Mangement Plan